

Xây dựng bộ dữ liệu 2.5D từ RELIS-3D phục vụ bài toán Path Planning

Report Group1 20260519

Le Loc and Tue Lam

Ngày 19 tháng 05 năm 2026

Tóm tắt nội dung

Báo cáo này trình bày quá trình xây dựng bộ dữ liệu 2.5D từ RELIS-3D nhằm phục vụ bài toán lập kế hoạch đường đi trong môi trường off-road. Dữ liệu đầu vào gồm point cloud LiDAR dạng `.bin` và semantic label dạng `.label`. Nhóm thực hiện chuyển đổi point cloud 3D sang lưới 2.5D, trong đó mỗi ô lưới lưu các thông tin gồm độ cao địa hình, nhãn ngữ nghĩa, độ dốc, độ gồ ghề và bản đồ chi phí di chuyển. Kết quả đầu ra được lưu dưới dạng `.npz` cho nhiều frame, đồng thời xuất các ảnh minh họa gồm elevation map, semantic map và costmap. Bộ dữ liệu 2.5D này có thể được sử dụng làm đầu vào cho các thuật toán path planning như A*, Dijkstra hoặc RRT*.

Mục lục

1	Giới thiệu bài toán	3
2	Tổng quan về RELLIS-3D	3
3	Mục tiêu xây dựng bộ dữ liệu 2.5D	4
4	Cấu trúc thư mục thực hiện	4
5	Phương pháp xây dựng dữ liệu 2.5D	5
5.1	Bước 1: Trích xuất và tiền xử lý dữ liệu	6
5.2	Bước 2: Xây dựng lưới độ cao 2.5D	6
5.3	Bước 3: Tích hợp semantic label và tạo costmap	8
6	Kết quả thực nghiệm	9
6.1	Kết quả elevation map	9
6.2	Kết quả semantic map	9
6.3	Kết quả costmap	11
7	Ứng dụng cho bài toán Path Planning	11
8	Demo trên RViz2	12
8.1	Các topic sử dụng	13
8.2	Lệnh build và chạy	13
8.3	Kết quả	14
9	Đánh giá và nhận xét	14
10	Kết luận	15

1 Giới thiệu bài toán

Path planning là một bài toán quan trọng trong lĩnh vực robot tự hành. Mục tiêu của bài toán là tìm một đường đi an toàn và hợp lý từ vị trí bắt đầu đến vị trí đích. Trong môi trường đô thị, robot thường di chuyển trên mặt đường tương đối bằng phẳng và có cấu trúc rõ ràng. Tuy nhiên, trong môi trường off-road, robot phải đối mặt với nhiều loại địa hình phức tạp như cỏ, bùn, nước, đá vụn, bụi cây, thân cây, hàng rào hoặc các vật cản tự nhiên.

Do đó, để robot có thể lập kế hoạch đường đi hiệu quả, dữ liệu đầu vào không chỉ cần biểu diễn vị trí vật cản mà còn cần thể hiện được đặc trưng địa hình. Một cách biểu diễn phù hợp là dữ liệu 2.5D. Biểu diễn 2.5D có thể hiểu là bản đồ 2D trên mặt phẳng $X - Y$, nhưng mỗi ô lưới có thêm thông tin về độ cao Z và các đặc trưng liên quan đến khả năng di chuyển.

Trong báo cáo này, nhóm sử dụng dữ liệu RELIS-3D để xây dựng bộ dữ liệu 2.5D. Mỗi frame LiDAR được chuyển thành một lưới 2.5D gồm:

- Elevation map: bản đồ độ cao.
- Semantic map: bản đồ nhãn ngữ nghĩa.
- Slope map: bản đồ độ dốc.
- Roughness map: bản đồ độ gồ ghề.
- Costmap: bản đồ chi phí di chuyển.
- Valid mask: mặt nạ biểu diễn ô có dữ liệu hợp lệ.

2 Tổng quan về RELIS-3D

RELIS-3D là một bộ dữ liệu đa cảm biến được thu thập trong môi trường off-road tại Relis Campus của Texas A&M University. Bộ dữ liệu này được thiết kế nhằm hỗ trợ các nghiên cứu về nhận thức ngữ nghĩa và điều hướng tự hành trong môi trường địa hình phức tạp.

Theo bài báo gốc [1], RELIS-3D bao gồm dữ liệu từ nhiều cảm biến như camera, LiDAR, GPS/IMU và các nhãn semantic cho ảnh cũng như point cloud. Trong phạm vi bài này, nhóm tập trung vào hai loại dữ liệu chính:

- Point cloud LiDAR dạng `.bin`.
- Semantic label dạng `.label`.

Mỗi điểm LiDAR trong file `.bin` có dạng:

$$p_i = (x_i, y_i, z_i, intensity_i) \quad (1)$$

Trong đó:

- x_i, y_i : tọa độ của điểm trên mặt phẳng ngang.
- z_i : độ cao của điểm.
- $intensity_i$: cường độ phản xạ của tia LiDAR.

File `.label` chứa nhãn semantic tương ứng với từng điểm trong point cloud. Nhãn semantic cho biết điểm đó thuộc loại đối tượng hoặc địa hình nào, ví dụ như đất, cỏ, cây, nước, bùn, hàng rào hoặc đường bê tông.

3 Mục tiêu xây dựng bộ dữ liệu 2.5D

Mục tiêu của nhóm là chuyển đổi dữ liệu LiDAR 3D của RELLIS-3D sang biểu diễn 2.5D phù hợp hơn cho bài toán path planning.

Dữ liệu đầu vào:

- File point cloud: `000000.bin`, `000001.bin`, ...
- File semantic label: `000000.label`, `000001.label`, ...

Dữ liệu đầu ra cho mỗi frame là một file `.npz`, chứa các thành phần:

Thành phần	Ý nghĩa
<code>elevation</code>	Bản đồ độ cao địa hình
<code>semantic</code>	Bản đồ nhãn ngữ nghĩa
<code>slope</code>	Bản đồ độ dốc
<code>roughness</code>	Bản đồ độ gồ ghề
<code>costmap</code>	Bản đồ chi phí di chuyển
<code>valid_mask</code>	Mặt nạ ô có dữ liệu hợp lệ

Bảng 1: Các thành phần trong file dữ liệu 2.5D đầu ra

4 Cấu trúc thư mục thực hiện

Trong quá trình thực nghiệm, nhóm tổ chức thư mục như sau:

```

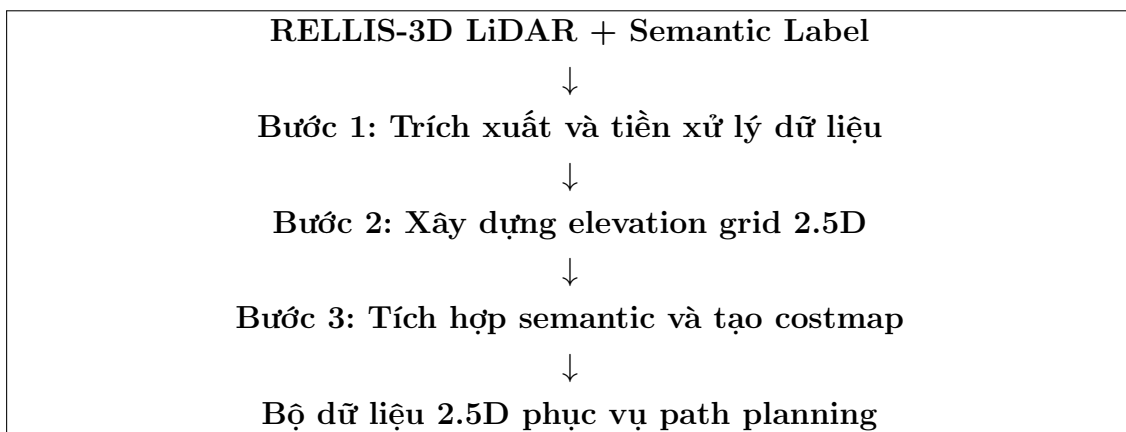
p25d/
|-- demo.py
|-- Rellis_3D_os1_cloud_node_kitti_bin/
|   |-- 000000.bin
|   |-- 000001.bin
|   |-- 000002.bin
|   '-- ...
|-- Rellis_3D_os1_cloud_node_semantickitti_label_id_20210614/
|   |-- 000000.label
|   |-- 000001.label
|   |-- 000002.label
|   '-- ...
'-- RELLIS_2_5D_output/
    |-- 000000.npz
    |-- 000001.npz
    |-- 000002.npz
    '-- preview/
        |-- 000000_elevation.png
        |-- 000000_semantic.png
        |-- 000000_costmap.png
        '-- ...

```

File `demo.py` được sử dụng để đọc nhiều frame LiDAR, ghép từng file `.bin` với file `.label` cùng tên, sau đó tạo dữ liệu 2.5D và lưu kết quả.

5 Phương pháp xây dựng dữ liệu 2.5D

Pipeline xử lý của nhóm gồm ba bước chính:



Hình 1: Pipeline tổng quát xây dựng dữ liệu 2.5D từ RELLIS-3D

5.1 Bước 1: Trích xuất và tiền xử lý dữ liệu

Ở bước đầu tiên, nhóm đọc dữ liệu point cloud từ file `.bin` và đọc nhãn semantic từ file `.label`. Với mỗi frame, file `.bin` và `.label` phải có cùng tên. Ví dụ:

```
000000.bin  <->  000000.label
000001.bin  <->  000001.label
```

Point cloud được đọc bằng kiểu dữ liệu `float32` và được reshape về dạng $N \times 4$:

```
points = np.fromfile(bin_path, dtype=np.float32).reshape(-1, 4)
```

Label được đọc bằng kiểu dữ liệu `uint32`. Phần semantic label được lấy bằng phép toán bit:

```
labels = np.fromfile(label_path, dtype=np.uint32)
semantic = labels & 0xFFFF
```

Sau đó, các điểm LiDAR được lọc theo vùng quan tâm quanh robot:

$$x \in [x_{min}, x_{max}], \quad y \in [y_{min}, y_{max}] \quad (2)$$

Trong thực nghiệm, nhóm sử dụng cấu hình:

Tham số	Giá trị
x_{min}	-10.0 m
x_{max}	30.0 m
y_{min}	-15.0 m
y_{max}	15.0 m
Resolution	0.2 m

Bảng 2: Cấu hình vùng quan tâm và độ phân giải lưới

Việc lọc vùng quan tâm giúp giảm số lượng điểm cần xử lý và tập trung vào khu vực có ý nghĩa đối với bài toán path planning.

5.2 Bước 2: Xây dựng lưới độ cao 2.5D

Sau khi lọc dữ liệu, nhóm chia mặt phẳng $X - Y$ thành các ô lưới vuông có kích thước cố định. Với mỗi điểm (x, y, z) , chỉ số ô lưới được tính như sau:

$$grid_x = \left\lfloor \frac{x - x_{min}}{resolution} \right\rfloor \quad (3)$$

$$grid_y = \left\lfloor \frac{y - y_{min}}{resolution} \right\rfloor \quad (4)$$

Với cấu hình:

$$resolution = 0.2m \quad (5)$$

kích thước lưới thu được là:

$$W = \frac{x_{max} - x_{min}}{resolution} = \frac{30 - (-10)}{0.2} = 200 \quad (6)$$

$$H = \frac{y_{max} - y_{min}}{resolution} = \frac{15 - (-15)}{0.2} = 150 \quad (7)$$

Do đó, mỗi frame được biểu diễn bằng một lưới có kích thước 150×200 .

Với mỗi ô lưới, nhóm lấy tất cả các điểm LiDAR rơi vào ô đó và tính elevation bằng phần trăm vị thứ 25 của các giá trị z :

$$elevation(i, j) = percentile(Z_{i,j}, 25) \quad (8)$$

Trong đó $Z_{i,j}$ là tập các giá trị z của các điểm nằm trong ô (i, j) .

Nhóm sử dụng percentile 25 thay vì giá trị lớn nhất vì trong môi trường off-road có nhiều đối tượng như cỏ, bụi cây hoặc vật cản. Nếu dùng giá trị lớn nhất, elevation có thể bị ảnh hưởng bởi các điểm không thuộc mặt đất.

Ngoài elevation, nhóm tính thêm roughness bằng độ lệch chuẩn của các giá trị z trong từng ô:

$$roughness(i, j) = std(Z_{i,j}) \quad (9)$$

Slope được tính từ gradient của elevation grid:

$$slope = \arctan \left(\sqrt{\left(\frac{\partial z}{\partial x} \right)^2 + \left(\frac{\partial z}{\partial y} \right)^2} \right) \quad (10)$$

5.3 Bước 3: Tích hợp semantic label và tạo costmap

Mỗi điểm LiDAR trong RELIS-3D có một nhãn semantic. Khi chuyển sang lưới 2.5D, mỗi ô có thể chứa nhiều điểm với nhiều nhãn khác nhau. Do đó, nhóm gán nhãn semantic cho từng ô bằng phương pháp majority voting, tức là chọn nhãn xuất hiện nhiều nhất trong ô.

Tuy nhiên, với các nhãn vật cản như cây, cột, nước, xe, người, hàng rào hoặc rào chắn, nhóm ưu tiên gán ô đó là vật cản nếu trong ô có xuất hiện một trong các nhãn này.

Các nhãn vật cản được sử dụng:

$$\{4, 5, 6, 8, 12, 15, 17, 18, 19, 27\} \quad (11)$$

Sau khi có semantic map, nhóm chuyển semantic label thành semantic cost. Bảng cost được thiết kế dựa trên khả năng di chuyển của robot trên từng loại địa hình.

Label ID	Loại địa hình / vật thể	Cost
0	void / unknown	150
1	dirt	20
3	grass	40
4	tree	255
5	pole	255
6	water	255
8	vehicle	255
10	asphalt	10
12	building	255
15	log	100
17	person	255
18	fence	255
19	bush	90
23	concrete	10
27	barrier	255
31	puddle	80
33	mud	70
34	rubble	90

Bảng 3: Bảng chuyển đổi semantic label sang chi phí di chuyển

Costmap cuối cùng được tính bằng cách kết hợp semantic cost, slope cost và roughness cost:

$$costmap = 0.5 \times semantic_cost + 0.3 \times slope_cost + 0.2 \times roughness_cost \quad (12)$$

Trong đó:

- `semantic_cost`: chi phí dựa trên nhãn địa hình.
- `slope_cost`: chi phí dựa trên độ dốc.
- `roughness_cost`: chi phí dựa trên độ gồ ghề.

Các ô thuộc vật cản được gán trực tiếp:

$$costmap(i, j) = 255 \quad (13)$$

Ý nghĩa của `costmap`:

- Cost thấp: vùng dễ di chuyển.
- Cost trung bình: vùng có thể đi nhưng cần cân nhắc.
- Cost cao: vùng khó đi hoặc nguy hiểm.
- Cost bằng 255: vùng không thể đi qua.

6 Kết quả thực nghiệm

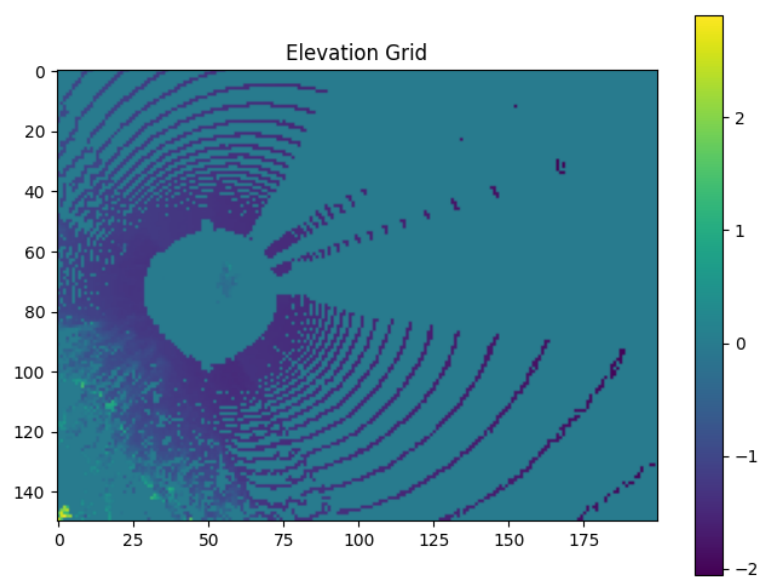
Nhóm đã chạy chương trình trên nhiều frame LiDAR của RELLIS-3D. Với mỗi frame hợp lệ, chương trình tạo ra một file `.npz` chứa đầy đủ các thành phần của dữ liệu 2.5D. Ngoài ra, chương trình lưu ảnh preview cho một số frame để kiểm tra trực quan.

6.1 Kết quả elevation map

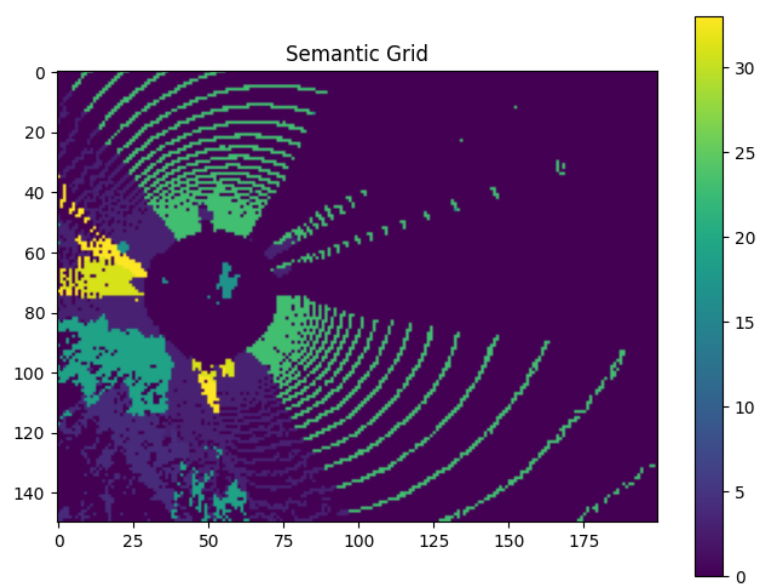
Elevation map thể hiện sự thay đổi độ cao của địa hình trong vùng quan tâm. Các vùng có giá trị khác nhau biểu diễn độ cao khác nhau của mặt đất hoặc vật thể.

6.2 Kết quả semantic map

Semantic map thể hiện nhãn ngữ nghĩa của từng ô lưới. Các giá trị khác nhau tương ứng với các loại địa hình hoặc vật thể khác nhau như cỏ, đất, cây, bùn, hàng rào hoặc vật cản.



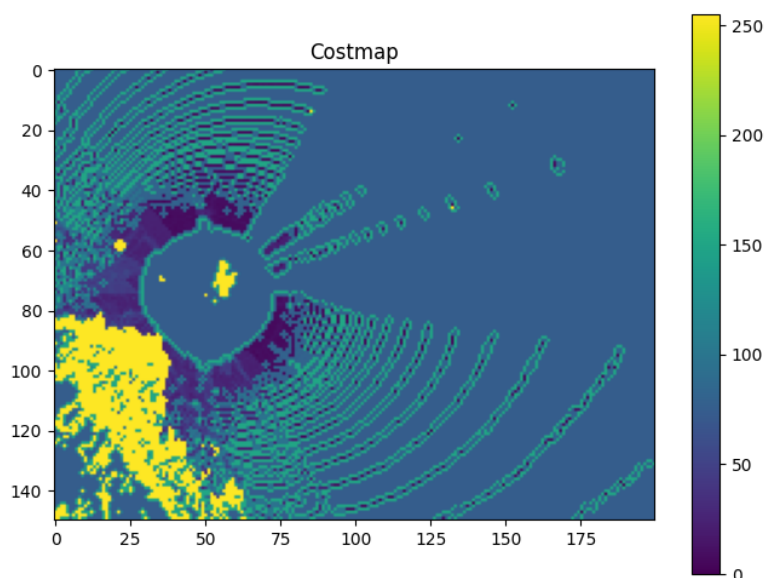
Hình 2: Kết quả elevation map của một frame LiDAR



Hình 3: Kết quả semantic map của một frame LiDAR

6.3 Kết quả costmap

Costmap là kết quả quan trọng nhất đối với bài toán path planning. Những vùng có chi phí thấp biểu diễn khu vực dễ đi, trong khi những vùng có chi phí cao biểu diễn khu vực khó đi hoặc không thể đi qua.



Hình 4: Kết quả costmap tạo từ elevation, semantic, slope và roughness

Từ các kết quả trên, có thể thấy dữ liệu point cloud 3D ban đầu đã được chuyển đổi thành biểu diễn 2.5D rõ ràng hơn. Biểu diễn này phù hợp để sử dụng trong các bài toán lập kế hoạch đường đi vì nó vừa giữ được thông tin hình học của địa hình, vừa tích hợp được thông tin ngữ nghĩa.

7 Ứng dụng cho bài toán Path Planning

Sau khi tạo costmap, dữ liệu này có thể được sử dụng làm đầu vào cho các thuật toán path planning. Trong đó, robot sẽ tìm đường đi từ điểm bắt đầu đến điểm đích sao cho tổng chi phí là nhỏ nhất.

Một số thuật toán có thể sử dụng costmap này gồm:

- A*
- Dijkstra
- RRT*

Với thuật toán A*, mỗi ô trong costmap được xem như một node trong đồ thị. Robot có thể di chuyển từ một ô sang các ô lân cận nếu ô đó không phải vật cản. Chi phí di chuyển phụ thuộc vào giá trị cost của ô.

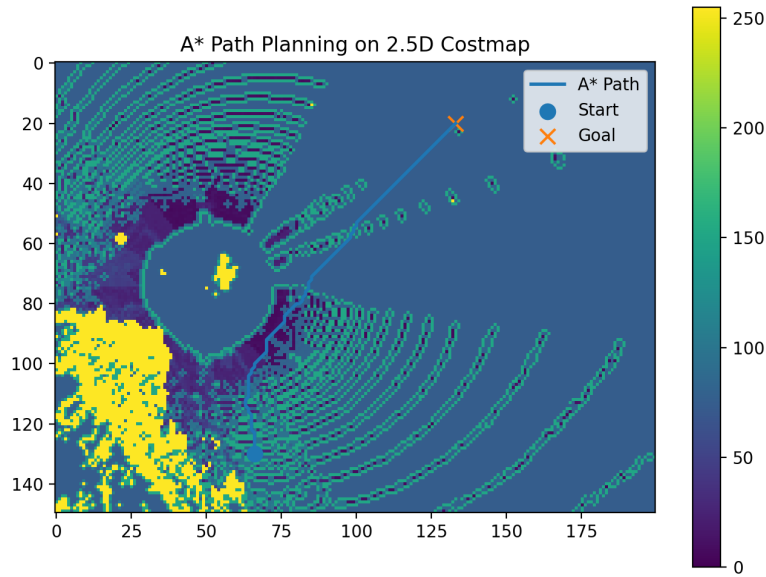
Hàm đánh giá trong A* có dạng:

$$f(n) = g(n) + h(n) \quad (14)$$

Trong đó:

- $g(n)$ là chi phí thực tế từ điểm bắt đầu đến node hiện tại.
- $h(n)$ là hàm heuristic ước lượng chi phí từ node hiện tại đến đích.
- $f(n)$ là tổng chi phí được dùng để chọn node tiếp theo.

Trong bài toán này, costmap giúp thuật toán tránh các vùng có chi phí cao như bùn, nước, đá vụn, bụi cây hoặc vật cản. Vì vậy, đường đi được tìm ra sẽ có xu hướng an toàn hơn so với việc chỉ sử dụng bản đồ 2D thông thường.



Hình 5: Kết quả chạy A* trên costmap 2.5D

8 Demo trên RViz2

Để trực quan hóa kết quả xây dựng dữ liệu 2.5D, nhóm thực hiện demo trên RViz2 trong môi trường ROS 2 Jazzy. File dữ liệu đầu vào là file .npz được tạo từ pipeline xử

lý RELIS-3D, trong đó có chứa `costmap`, `elevation`, `semantic`, `slope`, `roughness` và `valid_mask`.

8.1 Các topic sử dụng

Topic	Kiểu dữ liệu	Ý nghĩa
<code>/costmap_2d</code>	OccupancyGrid	Bản đồ chi phí 2.5D
<code>/astar_path</code>	Path	Đường đi A*
<code>/start_marker</code>	Marker	Điểm bắt đầu
<code>/goal_marker</code>	Marker	Điểm đích

Bảng 4: Các topic được publish để hiển thị trên RViz2

8.2 Lệnh build và chạy

Sau khi viết node, nhóm build package bằng lệnh:

```
cd ~/ros2_ws
source /opt/ros/jazzy/setup.bash
colcon build --packages-select rellis_25d_rviz --symlink-install
source ~/ros2_ws/install/setup.bash
```

Chạy node demo:

```
ros2 run rellis_25d_rviz show_costmap \
  --ros-args \
  -p npz_path:=/mnt/d/p25d/RELLIS_2_5D_output/000000.npz
```

Mở RViz2:

```
rviz2
```

Trong RViz2, đặt:

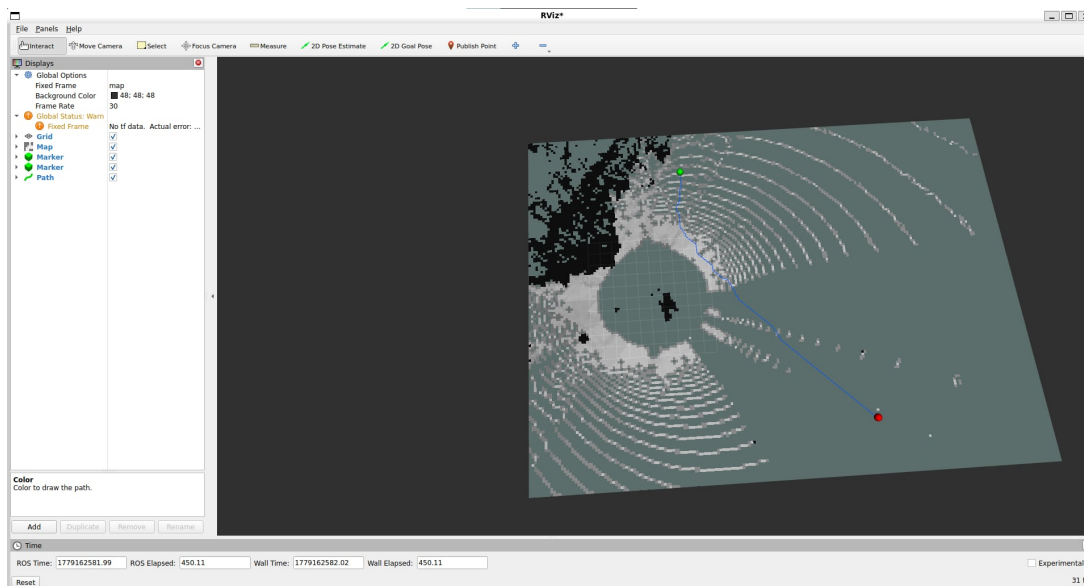
```
Fixed Frame = map
```

Sau đó thêm các display:

- Marker với topic `/colored_costmap`
- Marker với topic `/start_marker`
- Marker với topic `/goal_marker`
- Marker với topic `/astar_path`

8.3 Kết quả

Kết quả demo cho thấy costmap 2.5D được hiển thị trực quan trên RViz2. Vùng dễ đi, vùng khó đi và vật cản được biểu diễn bằng màu sắc khác nhau. Đường đi A* được vẽ từ điểm bắt đầu đến điểm đích, cho thấy costmap có thể được sử dụng làm đầu vào cho bài toán path planning.



Hình 6: Demo costmap 2.5D và đường đi A* trên RViz2

9 Đánh giá và nhận xét

Quá trình xây dựng dữ liệu 2.5D từ RELLIS-3D cho thấy phương pháp này có một số ưu điểm:

- Giảm độ phức tạp so với dữ liệu point cloud 3D đầy đủ.
- Giữ lại được thông tin quan trọng về độ cao địa hình.
- Tích hợp được nhãn ngữ nghĩa để phân biệt vùng dễ đi, khó đi và vật cản.
- Phù hợp với các thuật toán path planning sử dụng costmap.

Tuy nhiên, phương pháp hiện tại vẫn còn một số hạn chế:

- Một số ô lưới không có điểm LiDAR nên cần xử lý lỗ trống tốt hơn.
- Việc gán cost hiện tại chủ yếu dựa trên quy tắc thủ công.
- Elevation có thể bị ảnh hưởng bởi nhiễu hoặc các điểm thuộc vật thể không phải mặt đất.
- Chưa sử dụng pose để ghép nhiều frame thành bản đồ dày hơn.

Trong tương lai, có thể cải tiến bằng cách sử dụng nhiều frame liên tiếp, dùng pose để ghép point cloud, hoặc học costmap bằng mô hình học máy thay vì gán cost thủ công.

10 Kết luận

Trong báo cáo này, nhóm đã xây dựng một pipeline chuyển đổi dữ liệu RELIS-3D sang biểu diễn 2.5D phục vụ path planning. Dữ liệu đầu vào gồm point cloud LiDAR và semantic label. Qua các bước xử lý, nhóm tạo ra các bản đồ gồm elevation, semantic, slope, roughness, costmap và valid mask.

Kết quả thực nghiệm cho thấy pipeline có thể xử lý nhiều frame và lưu mỗi frame thành file `.npz`. Các ảnh preview như elevation map, semantic map và costmap giúp quan sát trực quan kết quả chuyển đổi. Costmap được tạo ra có thể sử dụng làm đầu vào cho các thuật toán path planning như A*, Dijkstra hoặc RRT*.

Như vậy, nhóm đã hoàn thành bước xây dựng bộ dữ liệu 2.5D từ RELIS-3D và chứng minh được khả năng ứng dụng của dữ liệu này trong bài toán lập kế hoạch đường đi cho robot tự hành trong môi trường off-road.

Tài liệu

- [1] Peng Jiang, Philip Osteen, Maggie Wigness, and Srikanth Saripalli. *RELIS-3D Dataset: Data, Benchmarks and Analysis*. arXiv:2011.12954, 2020. Available: <https://arxiv.org/abs/2011.12954>
- [2] Unmanned Systems Lab. *RELIS-3D: A Multi-modal Dataset for Off-Road Robotics*. GitHub repository. Available: <https://github.com/unmannedlab/RELIS-3D>